High-Speed CAVLC Encoder for 1080p 60-Hz H.264 Codec

Yongseok Yi and Byung Cheol Song, *Member, IEEE*

***Abstract*—In H.264/AVC and the variants, the coding of context-based adaptive variable length codes (CAVLC) requires demanding operations, particularly at high bitrates such as 100 Mbps. This letter presents two approaches to accelerate the coding operation substantially. Firstly, in the architectural aspect, we propose component-level parallelism and pipeline techniques capable of processing high-bitrate video data in a macroblock(MB)-level pipelined codec architecture. The second approach focuses on a specific part of the coding process, i.e., the residual block coding, in which the coefficient levels are coded without using look-up tables so we minimize the pertaining logic depth in the critical path, and we achieve higher operating clock frequencies. Additionally, two coefficient levels are processed in parallel by exploiting a look-ahead technique. The resulting architecture, merged in the MB-level pipelined codec system, is capable of coding up to 100 Mbps bitstreams in real-time, thus accommodating the real-time encoding of 1080p@60 Hz video.**

***Index Terms*—Baseline, CAVLC, entropy coding, H264, intra frame mode, level coding.**

I. Introduction

As flat-panel display size and spatiotemporal resolution of video data increase more and more, video codec should cope with higher coding bitrate. Also, digital AV streaming applications with very low coding latency, e.g., DTV interacted with game consoles, require intra-frame coding. For example, the bitrate for full HD (1080p@60 Hz) intra-frame coding can amount to 100 Mbps even with an H.264 encoder. Fortunately, most of the coding tools such as intra-prediction and transform meet the performance requirement of high bitrate since they may exploit possibilities of parallel processing and pipelining, at the cost of increased resources. However, those techniques can hardly be applied to the syntax processing part because *the syntax is sequential in nature*. The real-time processing of CAVLC [1] is computationally demanding, so it is inevitable to implement it in hardware.

Regarding the architectural improvement, there are several approaches such as zero skipping [2] and pipelining of the residual block coding on 4×4 block level [3]. The gain of the schemes is significant if MBs having almost no high-frequency coefficients are dominant in the input video data, and if the quantization parameter is large. In the opposite case, however, those approaches are insufficient to improve the performance.

Manuscript received December 17, 2007; revised May 20, 2008. The associate editor coordinating the review of this manuscript and approving it for publication was Dr. Yuriy V. Zakharov.

Y. Yi is with the Digital Media R&D Center, Samsung Electronics Co., Ltd., Suwon, Korea (e-mail:yongseok.yi@samsung.com).

B. C. Song is with the School of Electronic Engineering, Inha University, Incheon, Korea (e-mail:bcsong@inha.ac.kr).

Digital Object Identifier 10.1109/LSP.2008.2001982

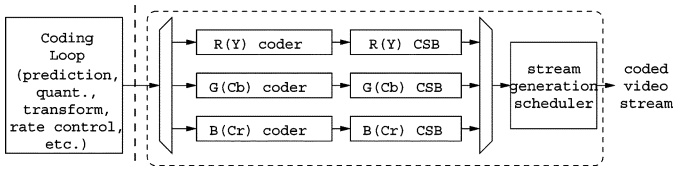

Fig. 1. Block diagram of the proposed architecture.

Increasing the operating clock frequency is a simple remedy. However, as the clock period shrinks, the CAVLC coder often encounters tight timing constraints due to the presence of a significant amount of look-up tables (LUTs). Since the use of LUTs introduces considerable circuit area and delay overhead, there have been many approaches to completely remove LUTs [4], [5], [6] or to at least reduce them [7]. However, those architectures still bring out a considerable delay; thereby, they are not suitable for high-end H.264 encoders supporting high bitrates or full HD resolution.

This work is about CAVLC for a full HD H.264 intra-frame codec that is pipelined on a MB-basis as in [8]. Since the codec system seeks low latency, it employs only an MB-row buffer without a frame buffer such as external DRAM, leading to a very tight performance constraint. Consequently, the syntax coder is constrained to consume less than only 400 clock cycles, hereinafter abbreviated to cc, per MB to achieve the real-time full HD encoding at a bitrate of up to 100 Mbps while the maximum clock frequency is 200 MHz. This letter presents a novel CAVLC architecture that meets the condition by processing the three color components in parallel, pipelining the actual coding and the stream processing, and replacing LUTs with arithmetic manipulations. Specifically, this letter focuses only on the syntax coding of nonzero coefficient levels that are not trailing ones, because coding of zero runs and trailing ones are relatively simple.

This letter is organized as follows. Section II presents the proposed component-parallel processing and task-level pipelining techniques at MB-level, and Section III describes the speed-up approaches for coefficient level processing in CAVLC. After the experimental data are shown in Section IV, the concluding remarks are made in Section V.

II. Component-Parallel Processing and Pipelining

Assume that the entire encoding process except the syntax coding, which is called coding loop here, is completed for every color component of RGB or YCbCr. The coding loop is composed of prediction, quantization, transform, etc., as in Fig. 1. The proposed architecture processes the components in parallel. Those three coders may or may not be identical depending on the elementary stream (ES) syntax structure and chroma format.

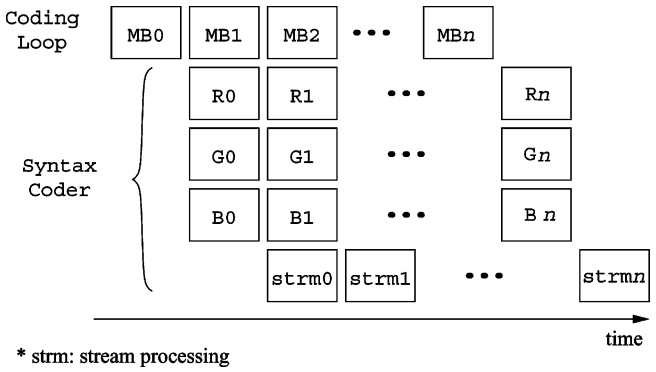

Fig. 2. MB-level pipelining of the syntax coder.

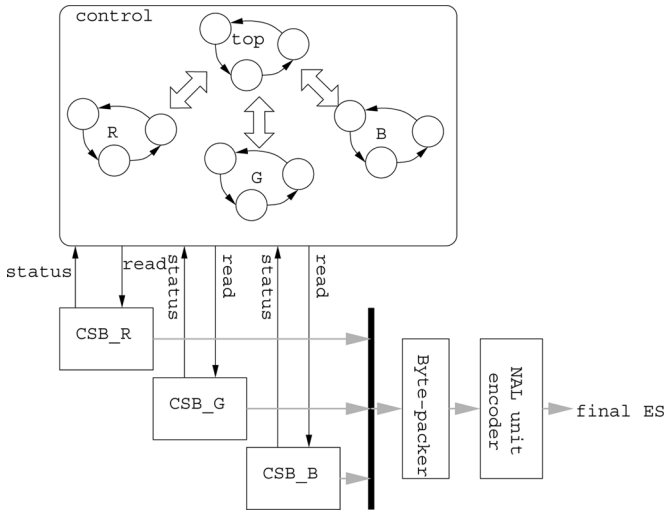

Fig. 3. Stream generation scheduler organization.

A syntax coder produces the corresponding *preliminary* ES for each color component of the given MB in parallel and stores the output in the following component stream buffer (CSB). Then, the stream generation scheduler combines the preliminary ES from the three buffers and puts the resulting (combined) ES into a network abstraction layer (NAL) unit, a container to deliver the video syntax over various network environments.

Assuming 1080p@60 Hz video, in order to generate a bitstream whose bitrate is as high as 100 Mbps, unfortunately, the coder for a color component alone *may* consume more than 400 cc, not leaving any room for the subsequent stream generation process. Although not dominant, this case often arises when processing a picture having some high frequency MBs, i.e., MBs including no small number of high-frequency coefficients due to complex texture. Since the low-latency requirement cannot tolerate even a few overtimed MBs, the path from the coder inputs to the final video stream out is pipelined. Partitioning the stages prior to the CSB is a natural choice here, leading to MB-level pipelining as depicted in Fig. 2. In this way, the syntax coder can safely process each MB data within the time constraint.

Each CSB is a double-buffer that enables seamless data flow. A syntax element or a set of syntax elements is written to the CSB in a form of a tuple (code, length), in which the code is a binary representation of the codeword and the length is arbitrary. Data written to the buffer are automatically byte-aligned except the last syntax element for the MB.

The stream generation scheduler comprises a hierarchical finite state machine (FSM) that controls the generation of the final ES, a byte-packer, and an NAL unit encoder as illustrated in Fig. 3. The top FSM initiates three sub-FSMs in R-G-B order, and a sub-FSM continues to transfer data from the CSB to the byte-packer until the buffer is empty. Since the size of the last data may not be a full byte, the byte packer uses an intermediate buffer to align the incoming data. Unlike the other color components, the sub-FSM for B receives a flag indicating whether the current MB is the last one of the picture, and, if reaching this MB, it appends a stopping bit to the last word being transferred. Stream concatenation is actually completed by the byte-packer and the resulting preliminary ES is processed by the NAL unit encoder that inserts start codes and emulation prevention bytes.

TABLE I
PROCEDURE TO CODE NONZERO COEFFICIENT LEVELS

Procedure CODE_NONZERO_LEVEL	
Inputs:	
n_{tot} := the number of nonzero coefficients	
n_{tr1} := the number of trailing ones	
x_j := nonzero levels, $0 \le j \le 15$	
Outputs:	
c_j := codewords for x_j	
1	$t = (n_{\mathrm{tot}} > 10 \text{ AND } n_{\mathrm{tr1}} < 3)\ ?\ 1 : 0$
2	$j = 0$
3	**while** $j < n_{\mathrm{tot}}$ **do**
4	c_j = CODE_LEVEL(x_j, t)
5	**if** $t == 0$ **then** $t = 1$
6	**if** ($\lvert x_j \rvert > 3 \cdot 2^{t-1}$ AND $t < 6$) **then** $t = t + 1$
7	$j = j + 1$
8	**endwhile**

III. ENCODING OF COEFFICIENT LEVELS

A. Coefficient Level Processing

For coding efficiency, CAVLC adopts seven structured VLCs, i.e., VLC0 to VLC6, to code coefficient levels [9]. The selection of a proper table depends on the number of nonzero coefficients, the number of trailing ones, and the size of the previously coded level value. Let $t \in \{0, 1, 2, 3, 4, 5, 6\}$ denote the index of the VLC table.

Table I depicts the abstract procedure to code the nonzero coefficient levels of a residual block. The sub-procedure CODE_LEVEL, which produces a codeword from a given nonzero coefficient level, is described in more detail as follows. The codeword of a nonzero coefficient level comprises a prefix and a conditionally existing suffix. The prefix whose value is equal to the number of its leading zeros is represented in a unary form, while the suffix has a binary form. Hence, coding a coefficient level can be restated as deriving a triple $(x_{\mathrm{pfx}}, x_{\mathrm{sfx}}, l_{\mathrm{sfx}})$ from a given coefficient level and t, where the

elements of the triple denotes the prefix value, the suffix value, and the suffix length, respectively.

In the following discussion, we omit the coding process in case of $t = 0$ because it is simply derived from the general coding process when t is not equal to 0.

Let x denote the value of the coefficient level to be coded. Instead of directly using x whose dynamic range is $[-32768,32767]$, we use its translated version x' with a dynamic range of $[0, 65545]$ by the following relation. Actually, x' is used as an intermediate representation of the coefficient levels, called *levelCode*

$$x' = \begin{cases} 2 \cdot (|x| - 1) & x \geq 0 \\ 2 \cdot (|x| - 1) + 1 & x < 0. \end{cases} \quad (1)$$

Additionally, l_{sfx} is uniquely determined from x_{pfx} and t by the following equations:

$$l_{\text{sfx}} = \begin{cases} x_{\text{pfx}} - 3, & \text{if } x_{\text{pfx}} \geq 15 \\ t, & \text{otherwise.} \end{cases} \quad (2)$$

The relation of x' and the tuple $(x_{\text{pfx}}, x_{\text{sfx}})$ for a given t is as follows:

$$x_{\text{sfx}} = \begin{cases} x' - 2^t \cdot x_{\text{pfx}}, & 0 \leq x_{\text{pfx}} < 15 \\ x' - 15 \cdot 2^t, & x_{\text{pfx}} = 15 \\ x' - 15 \cdot 2^t - 2^{x_{\text{pfx}}-4}, & 15 < x_{\text{pfx}} \leq 19 \end{cases} \quad (3)$$

where $x_{\text{sfx}} \in [0, 2^{l_{\text{sfx}}})$. If we can derive the unknown x_{pfx} from x' and t, the remaining unknowns x_{sfx} and l_{sfx} are also calculated by (2) and (3). We explain the derivation of x_{pfx} from x' and t throughout the rest of the subsection.

According to (3), x' can be partitioned into several non-overlapping subsets for $t \neq 0$. Note that the maximum of x_{pfx} is determined to 19 from the predefined dynamic range of x'. The first subset X_0 corresponds to $0 \leq x_{\text{pfx}} < 15$ and has the elements that are represented using a fixed suffix length ($l_{\text{sfx}} = t$), i.e.,

$$X_0 = \{x' | x' = 2^t \cdot x_{\text{pfx}} + x_{\text{sfx}} \\ 0 \leq x_{\text{pfx}} < 15, 0 \leq x_{\text{sfx}} < 2^t\}. \quad (4)$$

Each of the remaining subsets corresponds to an integer value of $x_{\text{pfx}} \geq 15$, i.e., $x_{\text{pfx}} = 14 + i$ for X_i, and are expressed as follows:

$$X_1 = \{x' | x' = 15 \cdot 2^t + x_{\text{sfx}}, 0 \leq x_{\text{sfx}} < 2^{12}\} \\ X_i = \{x' | x' = 15 \cdot 2^t + 2^{10+i} + x_{\text{sfx}}, 0 \leq x_{\text{sfx}} < 2^{11+i}\} \\ 1 < i \leq 5. \quad (5)$$

For a given t, we can locate one of the subsets defined by (4) and (5) that includes a given x'. So, a unique x_{pfx} corresponding to the specific subset is directly derived. For example, Table II lists X_i for $t = 2$. Now, to locate the subset including x', x' should be compared with the boundaries of X_i. However, from a hardware perspective, the comparisons require five subtractions and should be done one-by-one, introducing unnecessary logics and delay. To simplify the comparing operation, we derive x'' by subtracting the common term $15 \cdot 2^t$ from x' in (5). The modified subsets $X'_1, \ldots, X'_5$ are shown in Table III, except X_0. Since the subtraction of the common term is not applicable, the X_0 case is excluded in Table III, but it is examined implicitly because $x' \in X_0$ if $x' - 15 \cdot 2^t < 0$. Finally, we can find a proper subset, i.e., index i, by checking the four most significant bits of x'' as in Table III, which can be implemented using only a few gates with the logic depth of 3. As a result, we can derive a unique x_{pfx} from the selected index i. Note, in case of $x' \in X_0$, we can obtain a unique x_{pfx} from (4).

TABLE II
X_i FOR $t = 2$

X_i	i
$[0, 60)$	0
$[60, 4156)$	1
$[4156, 12348)$	2
$[12348, 28732)$	3
$[28732, 61500)$	4
$[61500, \infty)$	5

TABLE III
SUBSETS OF x''

X'_i	i
$[0, 2^{12})$	1
$[2^{12}, 2^{13})$	2
$[2^{13}, 2^{14})$	3
$[2^{14}, 2^{15})$	4
$[2^{15}, \infty)$	5

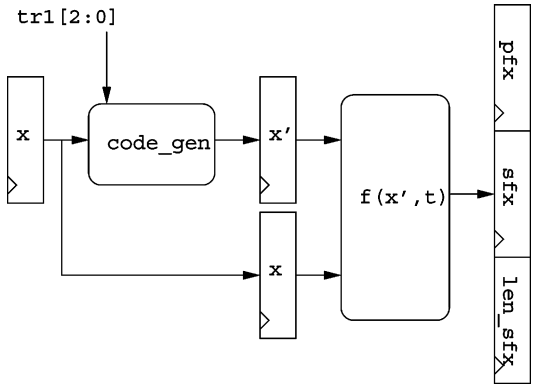

Fig. 4. Block diagram of CODE_LEVEL.

Fig. 4 describes the proposed architecture called CODE_LEVEL that derives the triple $(x_{\text{pfx}}, x_{\text{sfx}}, l_{\text{sfx}})$.

B. Parallel Processing of Coefficient Levels

Coding a 4 $\times$4 residual block is inherently a sequential process because the evaluation of each coefficient level depends on t, which is derived from the previously coded coefficient level. In spite of the dependency, however, two coefficients can be processed simultaneously by using the look-ahead technique.

Let t_j denote t used for x_j. Then, according to Table I, there are only two possible t values for x_j: t_{j-1} and $t_{j-1} + 1$, except the situation with $j = 0$, where x_1 can take three possible t values. Although the following look-ahead technique may be extended to accommodate all the cases, we only consider the cases of $j > 0$ here. Suppose that x_j and x_{j+1} are being processed in parallel, x_{j+1} is evaluated for both t_j and $t_j + 1$ in

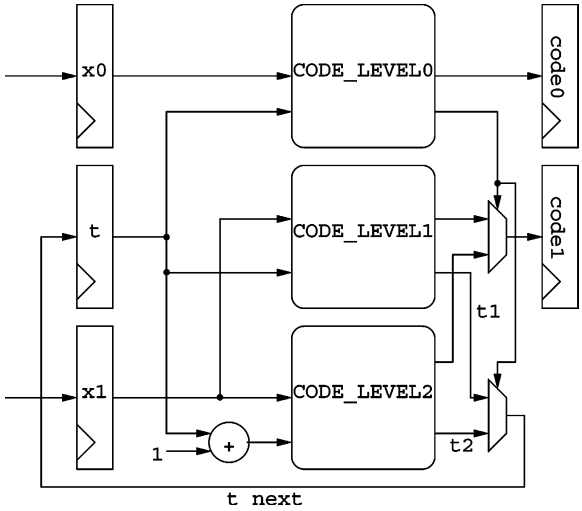

Fig. 5. Parallel processing of coefficient levels.

advance, and then one out of the two results is chosen based on the index generated by x_j.

Fig. 5 illustrates the logic diagram that realizes the parallel processing. Two input coefficients are passed as inputs to CODE_LEVELs before each cycle. Note that the parallel processing can substantially reduce the latency of a 4 $\times$ 4 residual block processing by 30% on the average.

IV. Implementation and Experimental Results

The proposed architecture has been described using Verilog HDL, synthesized using TSMC 90-nm standard CMOS technology, and is undergoing a fabrication process as a part of the low-latency H.264 codec system. The critical path delay of 3.1 ns arises from the coefficient coding logics due to stacked multiplexers. Considering clock uncertainty of 30%, the estimated maximum operating frequency amounts to 227 MHz. The total logic gate count for the proposed full HD CAVLC is 66 559. The on-chip SRAMs consist of three pairs of 8-Kb SRAMs, which correspond to three CSBs, respectively. For fair comparison between designs with different target resolutions, we use gate count per input data rate (GCDR), which is defined as (gate count) $\times$ (clock frequency)/(# of MBs/s). Table IV compares the proposed architecture with the previous designs [3]–[5]. As can be seen in the table, the proposed architecture has the lower GCDR factor and is much more suitable for systems with high specifications. Even though [4] provides lower GCDR than the proposed architecture, the architecture of [4] cannot support the full HD encoding outputting significantly higher bit-rate than CIF encoding inherently. Moreover, since the gate counts of [4] and [5] do not include the size of memory, their actual GCDRs may be larger than our estimates.

Table V shows the average encoding latency per MB for various test sequences. The test sequences in Table V have been used for H.264 standardization. Note that the average clock cycle count for all the test sequences is much smaller than the MB latency constraint of the codec system, i.e., 400 cc. Therefore, the proposed architecture is able to process 100 Mbps streams in real-time with the operating frequency of 200 MHz.

TABLE IV
Comparison of the CAVLC Encoder Architectures

	Logic (gates)	Mem (Kb)	Tech. (um)	Freq. (MHz)	Max. Spec.	GCDR
prop.	66K	48	0.09	200	1080p60 RGB4:4:4	45
[3]	23K	11	0.18	100	720p30 YCbCr4:2:0	55
[4]	6.9K	N/A	FPGA	50	CIF 30Hz YCbCr4:2:0	29
[5]	9.2K	N/A	0.35	28	QCIF 10Hz YCbCr4:2:0	254

TABLE V
Average MB Encoding Latency

Test Sequence	Resolution & frame rate	Bitrate (Mbps)	Latency (cc)
Freeway	1080p60	100	272
CrowdRun	1080p60	100	323
Waves	1080p60	100	299
OldTownCross	1080p60	100	270
DucksTakeOff	1080p24	100	185
InToTree	1080p24	100	175

V. Conclusion

This letter presented a high-performance H.264/AVC syntax coding architecture that can encode video streams with bitrate as high as 100 Mbps. The techniques to achieve the required performance include 1) component-wise parallel encoding; 2) pipelining the MB-level tasks, i.e., the coding stage and the stream generation stage; 3) arithmetic derivation of coefficient level codewords to speed-up the symbol coding rate and to eliminate the use of LUTs; and 4) parallel processing of coefficient levels.

The experimental results for various full HD video sequences showed that the proposed architecture is capable of processing the high bitrates video data in real-time. Finally, the proposed CAVLC syntax coder has been integrated into a 1080p@60 Hz H.264 intra-frame codec.

References

[1] ITU-T, H.264, Advanced Video Coding for Generic Audiovisual Services, 2005.
[2] T. Tsa, D. Fang, and Y. Pan, "A hybrid CAVLD architecture design with low complexity and low power considerations," in *Proc. IEEE Int. Conf. Multimedia and Expo*, Jul. 2–5, 2007, pp. 1910–1913.
[3] T. Chen, Y. Huang, C. Tsai, B. Hsieh, and L. Chen, "Architecture design of context-based adaptive variable-length coding for H.264/AVC," *IEEE Trans. Circuits Syst. II*, vol. 53, no. 9, pp. 832–836, Sep. 2006.
[4] C. Rahman and W. Badawy, "CAVLC encoder design for real-time mobile video applications," *IEEE Trans. Circuits Syst. II*, vol. 54, no. 10, pp. 873–877, Oct. 2007.
[5] Y. Lai, C. Chou, and Y. Chung, "A simple and cost effective video encoder with memory-reducing CAVLC," in *Proc. IEEE Int. Symp. Circuits and Systems*, May 23–26, 2005, vol. 1, pp. 432–435.
[6] C. Chien, K. Lu, Y. Shih, and J. Guo, "A high performance CAVLC encoder design for MPEG-4 AVC/H.264 video coding applications," in *Proc. IEEE Int. Symp. Circuits and Systems*, May 21–24, 2006, pp. 3838–3841.
[7] Y. Lin and P. Chen, "An efficient implementation of CAVLC for H.264/AVC," in *Proc. IEEE Int. Conf. Innovative Computing 2006*, Aug. 30–01, 2006, vol. 3, pp. 601–604.
[8] T. Chen, Y. Huang, and L. Chen, "Analysis and design of macroblock pipelining for H.264/AVC VLSI architecture," in *Proc. IEEE Int. Symp. Circuits and Systems*, May 23–26, 2004, vol. 2, pp. 273–276.
[9] G. Bjontegaard and K. Lillevold, "Context-adaptive VLC (CVLC) coding of coefficients," JVT Document JVT-C028. Fairfax, VA, 2002.